

Cálculo Paralelo do Conjunto de Mandelbrot com MPI + OpenMP

Versão híbrida: MPI Coordenador/Trabalhador entre nós e *workpool* OpenMP (sem mestre) dentro do nó

Bernardo Luiz Padilha Zamin • João Pedro Sostruznik Sotero da Cunha
Programação Paralela — Trabalho 4 (MPI + OpenMP), PUCRS — Escola Politécnica, 2026

1. Introdução

Implementa-se, em C, a versão **híbrida MPI + OpenMP** do cálculo do **conjunto de Mandelbrot** (imagem 7680×4320), reaproveitando o *Coordenador/Trabalhador* do grupo. São **dois níveis de paralelismo**: o **MPI** distribui blocos de linhas entre os **nós** (**um processo pesado por nó**) e o **OpenMP** usa os núcleos *dentro* de cada nó no modelo **workpool sem mestre**. Avaliam-se *speed-up*, eficiência e escalabilidade **forte** e **fraca** em até 4 nós, o efeito do **Hyper-Threading**, as **formas de alocar o coordenador** e a **comparação com a MPI pura**.

2. Modelo híbrido

Entre nós (MPI). Modelo **Coordenador/Trabalhador idêntico** ao da MPI pura, preservando a comparabilidade. O rank 0 distribui **blocos de rows_per_task linhas sob demanda**: o trabalhador pede (TAG_REQUEST), recebe TAG_TASK (tarefa) ou TAG_STOP (fim), calcula e devolve (TAG_RESULT). Há **um processo pesado por nó**, com todos os núcleos do nó à disposição das threads. Para cada pixel itera-se $z \leftarrow z^2 + c$ até $|z| > 2$ ou `max_iter`; pontos *internos* custam `max_iter` e *externos* terminam cedo, gerando **carga irregular** que exige **balanceamento dinâmico**.

3. Workpool OpenMP sem mestre

Cada bloco é processado por uma **região paralela** (`#pragma omp parallel`). O controle está numa **estrutura de dados compartilhada** `WorkPool{next,total}`: **todas as threads** executam o mesmo laço e **retiram atômica** a próxima linha (`#pragma omp atomic capture`) até esvaziar o pool — **sem thread mestre**, com escalonamento *self-service* emergindo da estrutura de dados (modelo *workpool/bag-of-tasks*). A granularidade é de **uma linha** (7680 px), que amortiza a sincronização e ainda equilibra a carga. Usa-se `MPI_Init_thread` (`MPI_THREAD_FUNNELED`; só a thread principal chama MPI) e `OMP_NUM_THREADS=16` (8 núcleos $\times$ 2 HT).

4. Alocação do coordenador

O coordenador só distribui/coleta, ficando **quase ocioso**. Há duas alocações, escolhidas **sem alterar o código** (via SLURM, `-n 4` vs `-n 5`): *dedicada* — ocupa um nó inteiro (em 4 nós sobram 3 trabalhadores e o nó do coordenador fica ocioso); e *compartilhada* — divide o nó com um trabalhador (4 trabalhadores em 4 nós).

5. Metodologia

Até 4 nós da Atlantica (LAD), SLURM, Xeon E5520, 16 CPUs lógicas/nó (8 núcleos $\times$ 2 HT). `ladcomp -env mpicc -O3 -fopenmp`. Tempo por MPI_Wtime antes do I/O. Baseline $T(1) = 93,19$ s (sequencial, `max_iter=2000`). **Forte**: `max_iter` fixo, variando recursos. **Fraca**: `max_iter=1500` $\times$ `workers`. $\text{Speed-up} = T(1)/T(p)$; $\text{Eficiência} = \text{Speed-up}/p$. Saída **byte-idêntica** à sequencial (md5).

6. Resultados e análise

Workpool (Tab. 1, Fig. a). Escala quase linear até os 8 núcleos físicos ($7,34\times$, 92%) e chega a **12,70** $\times$ com 16 threads — ganho de $\sim 73\%$ via **Hyper-Threading**, com *todas* as threads ocupadas apesar da carga irregular.

Escalabilidade forte (Tab. 3, Fig. b). O híbrido atinge **31** $\times$ em 4 nós. A eficiência cai em 4 nós (0,65) porque o tempo já é de ~ 3 s e o **custo fixo** — MPI e a **coleta da imagem de ~ 127 MB** no coordenador ($O(N)$, independente de p) — domina (Amdahl); com carga maior aproxima-se do ideal.

Alocação do coordenador (Fig. c, Tab. 4). Dedicar um nó ao coordenador ocioso **desperdiça 1/4 dos recursos**; *compartilhar* (4 trabalhadores) reduz o tempo de **2,96 para 2,35 s** — **20,7% mais rápido**.

Híbrido $\times$ MPI pura (Tab. 4). Nesta escala *compute-bound*, a **MPI pura com HT** (64 proc., todos os 4 nós calculando) é a mais rápida (**2,00 s, 46,5** $\times$); o híbrido *compartilhado* fica competitivo (2,35 s, 39,7 $\times$) e o *dedicado* perde por ociosar um nó. As vantagens do híbrido — menos *ranks*, menos memória e mensagens — surgem em **escalas maiores** ou com mais comunicação.

Escalabilidade fraca (Tab. 2, Fig. d). Com carga proporcional aos trabalhadores, o tempo permanece **praticamente constante** (5,1–5,4 s), com eficiência **97–104%** — próximo do ideal.

7. Conclusão

A versão cumpre o modelo: **MPI Coordenador/Trabalhador** (um processo por nó) + **workpool OpenMP sem mestre** (controle por estrutura de dados, uma thread por núcleo lógico, HT). O *workpool* escala bem no nó (12,7 $\times$) e a escalabilidade **fraca** é quase ideal. **Dedicar um nó ao coordenador é desvantajoso** (compartilhar é 20,7% melhor). Em pequena escala *compute-bound* a **MPI pura** vence em tempo bruto; o híbrido fica competitivo ao compartilhar o coordenador e tende a ganhar em escalas maiores.

Tabelas e gráfico

Tabela 1 — Workpool OpenMP (1 nó)

Threads	Tempo (s)	Speed-up	Efic.
1	87,01	1,00	1,00
2	43,68	1,99	1,00
4	23,40	3,72	0,93
8	11,85	7,34	0,92
16	6,85	12,70	0,79

Tabela 2 — Escalabilidade fraca

Nós	max_iter	Tempo (s)	Efic.
2 (1 w.)	1500	5,25	1,00
3 (2 w.)	3000	5,06	1,04
4 (3 w.)	4500	5,40	0,97

Tabela 3 — Escalabilidade forte: híbrido × MPI pura (HT)

nós	Híbrido (1 proc/nó, 16 threads)			MPI pura (16 proc/nó)		
	Tempo (s)	Speed-up	Efic.	Tempo (s)	Speed-up	Efic.
2	7,08	13,2	0,82	3,35	27,8	0,87
3	3,45	27,0	0,84	2,74	34,1	0,71
4	2,96	31,5	0,66	2,00	46,5	0,73

Tabela 4 — Comparação em 4 nós (alocação do coordenador e híbrido × MPI)

Configuração	Processos	Tempo (s)	Speed-up
MPI pura (HT, todos os nós calculando)	64	2,00	46,5
Híbrido — coordenador <i>compartilhado</i>	4 nós (4 w.)	2,35	39,7
Híbrido — coordenador <i>dedicado</i>	4 nós (3 w.)	2,96	31,5
MPI pura (1 processo por núcleo físico)	32	3,20	29,1

$T(1) = 93,19$ s (sequencial, 1 núcleo, max_iter=2000); Speed-up = $T(1)/T(p)$; Eficiência = Speed-up/ p , p = núcleos lógicos de cálculo (híbrido: trabalhadores×16; MPI pura: n° de processos). Imagem 7680×4320. Fraca: eficiência = $T(1 \text{ trab.})/T(p)$.

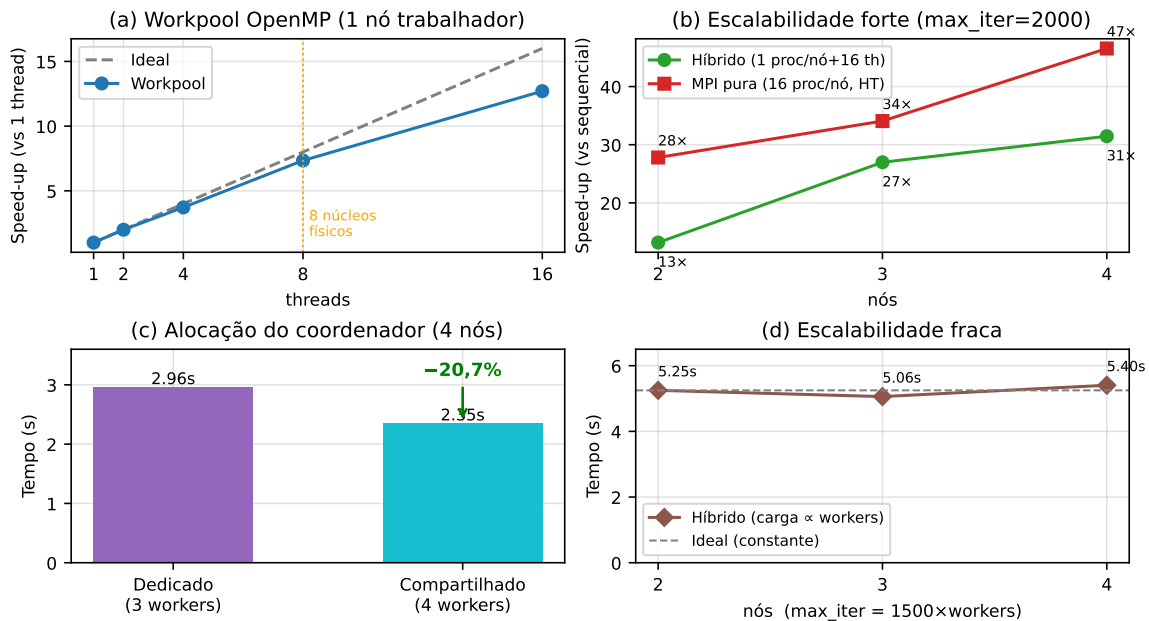


Figura 1 — (a) *workpool* OpenMP dentro de um nó (speed-up × threads); (b) escalabilidade forte, híbrido × MPI pura; (c) alocação do coordenador; (d) escalabilidade fraca.

Anexo — Código-fonte (mandelbrot-hib.c, trechos relevantes)

Versão híbrida MPI + OpenMP. O coordenador (rank 0) distribui blocos de linhas sob demanda e coleta os resultados (idêntico ao mandelbrot-mpi.c); cada trabalhador calcula seu bloco com o *workpool* OpenMP (compute_task_workpool, sem mestre). As funções inalteradas (mandelbrot, save_ppm) e o laço do coordenador são omitidos por brevidade. Compilação: `ladcomp -env mpicc -O3 mandelbrot-hib.c -o mandelbrot-hib -fopenmp -lm`.

```
1 #include <mpi.h>
2 #include <omp.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <math.h>
6 #define WIDTH 7680
7 #define HEIGHT 4320
8 #define DEFAULT_ROWS_PER_TASK 64
9 #define TAG_REQUEST 0
10 #define TAG_TASK 1
11 #define TAG_RESULT 2
12 #define TAG_STOP 3
13 typedef struct { int start_row; int num_rows; } Task;
14 /* Estrutura de dados compartilhada que CONTROLA o workpool (sem mestre):
15    next = índice da próxima linha do bloco ainda não tomada. */
16 typedef struct { int next; int total; } WorkPool;
17 int mandelbrot(double real, double imag, int max_iter); /* z=z^2+c; igual ao seq/MPI */
18 void save_ppm(const char *f, int *img, int max_iter); /* igual ao seq/MPI */
19 /* ===== WORKPOOL OpenMP: todas as threads trabalham, SEM MESTRE ===== */
20 void compute_task_workpool(const Task *task, int *buffer, int max_iter)
21 {
22     WorkPool pool = { .next = 0, .total = task->num_rows };
23     #pragma omp parallel /* todas as threads entram e trabalham */
24     {
25         while (1) {
26             int row;
27             #pragma omp atomic capture /* retira atômica a próxima linha */
28             { row = pool.next; pool.next++; } /* do pool compartilhado (sem mestre) */
29             if (row >= pool.total) break; /* pool vazio: esta thread termina */
30             int y = task->start_row + row;
31             for (int x = 0; x < WIDTH; x++) {
32                 double real = -2.5 + (3.5 * x / WIDTH);
33                 double imag = -1.0 + (2.0 * y / HEIGHT);
34                 buffer[row * WIDTH + x] = mandelbrot(real, imag, max_iter);
35             }
36         }
37     }
38 }
39 int main(int argc, char *argv[])
40 {
41     int max_iter = atoi(argv[1]);
42     int rows_per_task = (argc >= 3) ? atoi(argv[2]) : DEFAULT_ROWS_PER_TASK;
43     int provided; /* so a thread principal chama MPI */
44     MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
45     int rank, size;
46     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
47     MPI_Comm_size(MPI_COMM_WORLD, &size);
48     if (rank == 0) {
49         /* COORDENADOR: distribui blocos sob demanda (TAG_TASK) e coleta os
50            resultados (TAG_RESULT); cronometra (MPI_Wtime) e grava a imagem.
51            Idêntico ao mandelbrot-mpi.c (laço MPI_Probe / MPI_Recv / MPI_Send). */
52     } else {
53         /* TRABALHADOR: pede tarefa, calcula com o workpool, devolve. */
54         MPI_Status st; int request = 1;
55         while (1) {
56             MPI_Send(&request, 1, MPI_INT, 0, TAG_REQUEST, MPI_COMM_WORLD);
57             Task task;
58             MPI_Recv(&task, sizeof(Task), MPI_BYTE, 0, MPI_ANY_TAG,
59                     MPI_COMM_WORLD, &st);
60             if (st.MPI_TAG == TAG_STOP) break;
61             int *buffer = malloc(sizeof(int) * task.num_rows * WIDTH);
62             compute_task_workpool(&task, buffer, max_iter); /* <= OpenMP */
63             MPI_Send(&task, sizeof(Task), MPI_BYTE, 0, TAG_RESULT, MPI_COMM_WORLD);
64             MPI_Send(buffer, task.num_rows * WIDTH, MPI_INT, 0, TAG_RESULT,
65                     MPI_COMM_WORLD);
66             free(buffer);
67         }
68     }
69     MPI_Finalize();
70     return 0;
71 }
```